

Java Code Analysis!?! – Weak JWT Signing Key & Broken Access Control

1. Challenge Information

- Platform: **picoCTF (2023)**
- Category: **Web Exploitation**
- Difficulty: **Medium**
- Challenge Name: **"Java Code Analysis!?!"**
- Vulnerability Type: **Hardcoded JWT Signing Key → Broken Access Control / Privilege Escalation**
- Primary Issues: **Insecure fallback in JWT secret key generation & missing object-level access control (IDOR via userId)**

Related Standards:

- OWASP A02:2021 - Cryptographic Failures
- OWASP A01:2021 - Broken Access Control
- CWE-798: Use of Hard-coded Credentials
- CWE-639: Authorization Bypass Through User-Controlled Key

2. Executive Summary

BookShelf Pico is an online book-reading service that gates content behind three access tiers: Free, Premium, and Admin. Authentication and authorization are handled through a **Bearer JSON Web Token** (JWT) containing the claims `userId`, `email`, and `role`, signed with the HMAC256 algorithm.

Source code review reveals that the signing key (`SECRET_KEY`) is produced by the `SecretGenerator` class: the application first tries to read the key from a `server_secret.txt` file on disk, but if that file does not exist - the default state of a fresh challenge instance - it falls back to `generateRandomString(32)`, a function that despite its name **always returns the fixed string "1234"** instead of a genuinely random value. Because HMAC256 is a symmetric algorithm (the signing key and the verification key are the same), knowledge of "1234" allows anyone to forge an arbitrary, validly-signed JWT.

Simply changing the role claim, however, is not sufficient: `BookPdfAccessCheck` does not trust the role value embedded in the JWT directly - it **re-derives the user's real role** from the database using `userId`. To achieve privilege escalation, an attacker must therefore forge a JWT whose `userId` points to an account with the Admin role in the database, and sign that token with the recovered "1234" key. This ultimately grants access to every book tier, including the Admin-only "Flag" book.

3. Reconnaissance & Enumeration

Login & Functional Walkthrough:

- Using the provided credentials (`user / user`), the BookShelf Pico home page is reached. The UI lists books across three tiers - Free, Premium, Admin - but the test account can only open the Free-tier book.

getServerSecret() attempts to read the key from a server_secret.txt file next to the running jar. If that read fails (IOException) - the default state of a fresh challenge container - the application generates a new key via generateRandomString(32). That function, however, is not random at all:

```
private String generateRandomString(int len) {  
    // not so random  
    return "1234";  
}
```

This makes it reasonable to assume, with high confidence, that the effective SECRET_KEY on the live challenge server is the string "1234".

```
@Service  
class SecretGenerator {  
    private Logger logger = LoggerFactory.getLogger(SecretGenerator.class);  
    private static final String SERVER_SECRET_FILENAME = "server_secret.txt";  
  
    @Autowired  
    private UserDataPaths userDataPaths;  
  
    private String generateRandomString(int len) {  
        // not so random  
        return "1234";  
    }  
  
    String getServerSecret() {  
        try {  
            String secret = new String(FileOperation.readFile(userDataPaths.getCurrentJarPath(), SERVER_SECRET_FILENAME), Charset.defaultCharset());  
            logger.info("Server secret successfully read from the filesystem. Using the same for this runtime.");  
            return secret;  
        } catch (IOException e) {  
            logger.info(SERVER_SECRET_FILENAME + " file doesn't exists or something went wrong in reading that file. Generating a new secret for the server.");  
            String newSecret = generateRandomString(32);  
            try {  
                FileOperation.writeFile(userDataPaths.getCurrentJarPath(), SERVER_SECRET_FILENAME, newSecret.getBytes());  
            } catch (IOException ex) {  
                ex.printStackTrace();  
            }  
            logger.info("Newly generated secret is now written to the filesystem for persistence.");  
            return newSecret;  
        }  
    }  
}
```

Step 3: First Attempt - Changing Only the role Claim

Using jwt.io, the current userId is kept unchanged (userId = 1, the Free account) while role is changed to "Admin", and the token is re-signed with the "1234" key. This forged token is then used to request /base/books/pdf/4 (Premium).

Result: "You are trying to access a resource but you don't have the permission" - the request is still denied, even though the HMAC256 signature itself is accepted (no signature-verification error is returned).

In other words, the role claim in the token is largely cosmetic; the real trust boundary is `userId`. Because the JWT is signed with a key that has been recovered, an attacker can set `userId` to any value and re-sign a valid token, which is equivalent to impersonating any user in the system.

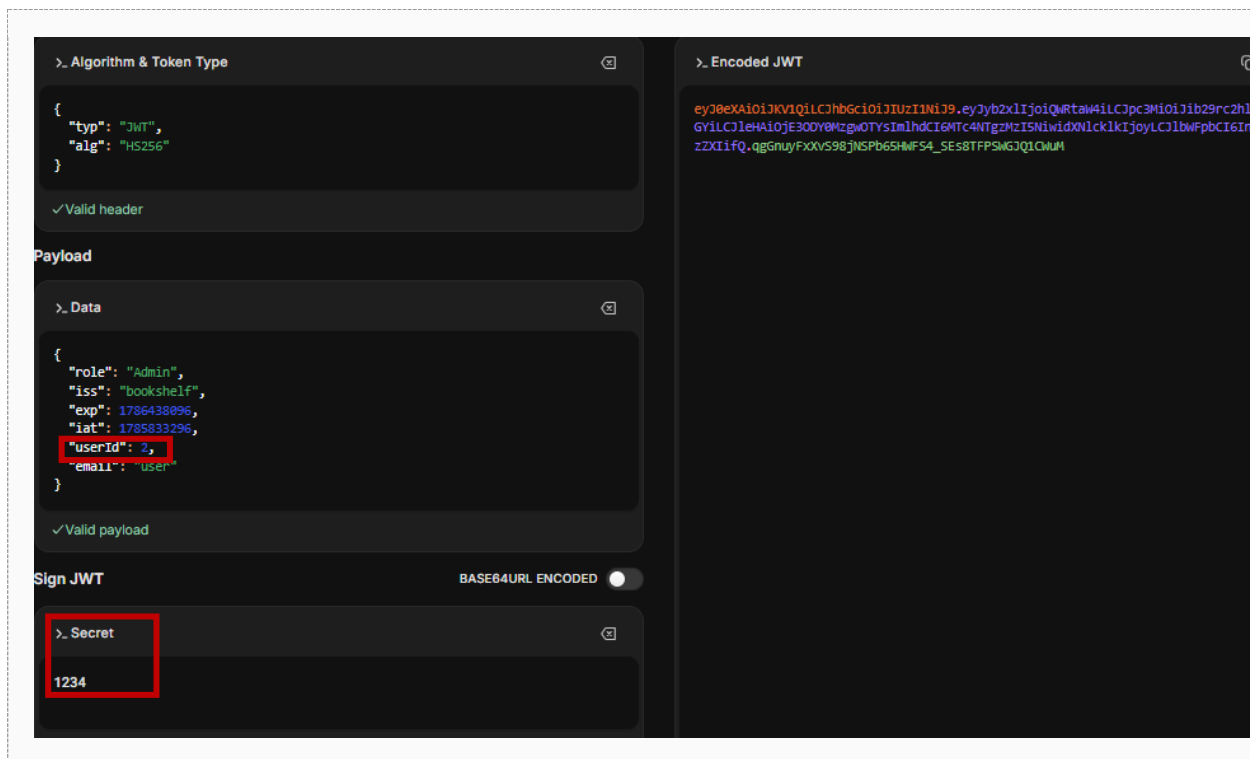
```
15 @Service
16 public class BookPdfAccessCheck {
17     private Logger logger = LoggerFactory.getLogger(BookPdfAccessCheck.class);
18
19     @Autowired
20     private BookRepository bookRepository;
21
22     @Autowired
23     private UserRepository userRepository;
24
25     public boolean verify(Integer bookId, Integer userId) throws ResourceNotFoundException {
26         Optional<Book> bookOptional = bookRepository.findById(bookId);
27         if(!bookOptional.isPresent()){
28             throw new ResourceNotFoundException("book with id: "+bookId+" not found");
29         }
30         Optional<User> userOptional = userRepository.findById(userId);
31         if(!userOptional.isPresent()){
32             throw new ResourceNotFoundException("user with id: "+userId+" not found");
33         }
34         Book book = bookOptional.get();
35         User user = userOptional.get();
36         boolean permissionGranted = (user.getRole().getValue())>=book.getRole().getValue();
37         logger.info("Book ID: "+bookId+"; User ID: "+userId+"; Permission Granted: "+permissionGranted);
38         return permissionGranted;
39     }
40 }
```

Step 5: Identifying the Admin's `userId` and Forging the Final Token

Since the supplied "user" account has `userId = 1` (Free) and IDs in the system are typically assigned sequentially, the Admin account is likely to hold the next low `userId` (`userId = 2`). A new JWT is crafted with the payload:

```
{
  "iss": "bookshelf",
  "userId": 2,
  "email": "user",
  "role": "Admin",
  "iat": <now>,
  "exp": <now + 7 days>
}
```

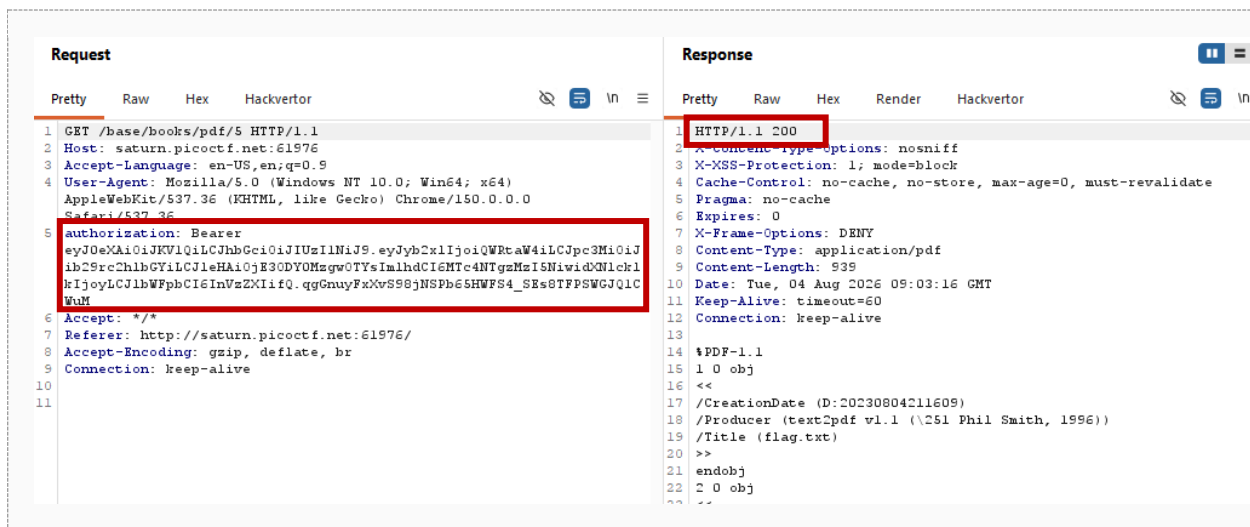
and signed with the HMAC256 key "1234" (done directly on jwt.io, or via a short script).



Step 6: Replacing the Token and Accessing the Restricted Resources

The new token is placed into the browser's localStorage / Authorization header (a reload or re-login may be needed for the new token to take effect), then the following endpoints are requested:

- GET /base/books/pdf/4 → Successfully opens the Premium-tier book.
- GET /base/books/pdf/5 → Successfully opens the Admin-tier book - the "Flag" book itself.



The returned PDF is opened and read to retrieve the challenge flag.

```

38 <<
39 /Length 80 R
40 >>
41 stream
42 BT
43 /F1 20 Tf
44 1 0 0 1 50 752 Tm
45 12 TL
46 (Great job! Here's your flag:)'
47 ()'
48 ()'
49 (picoCTF{w34k_jwt_n0t_g00d_7745dc02})'
50 ET
51 endstream
52 endobj
53 8 0 obj
54 119
55 endobj
56 3 0 obj
57 <<
58 /Type /Pages
59 /Count 1
--

```

5. Root Cause Analysis

- **Hardcoded / Predictable JWT Secret Fallback (CWE-798):** The fallback path used when `server_secret.txt` cannot be read produces a fixed key "1234" instead of a securely generated random value (e.g. via `SecureRandom`), which nullifies the security guarantees of HMAC256 signing entirely under this condition.
- **Broken Access Control via Client-Controlled Identifier (CWE-639):** Because the JWT is self-contained and can be freely re-signed, the server trusting the `userId` taken directly from the token - without binding it to an independently authenticated server-side session - is equivalent to letting the client declare its own identity, enabling full impersonation of any account.
- **Redundant / Misleading Role Claim:** The role claim in the JWT is not actually consulted by `BookPdfAccessCheck` for the access decision (the real role is re-queried from the database via `userId`), creating a false sense of security - a tester who only edits role, as in the first attempt above, will be misled into thinking the check is unreachable.
- **No Binding to a Valid Server-Side Session:** Tokens are valid for 7 days with no revocation mechanism or server-side allow-list, so a single forged token can be reused for the entire validity window.

6. Remediation & Best Practices

- **Generate a secure signing key:** Use `java.security.SecureRandom` to generate a sufficiently long key (at least 256 bits for HMAC256), and store it in a secrets manager (Vault, AWS Secrets Manager, a protected environment variable) rather than a plaintext file next to the application jar.
- **Fail closed, not open:** If the key cannot be read from a secure store, the application should refuse to start rather than silently generating a weak fallback key and continuing to run.

```

SecureRandom random = new SecureRandom();
byte[] bytes = new byte[32];

```



```
random.nextBytes(bytes);  
String secret = Base64.getEncoder().encodeToString(bytes);
```

- **Do not fully trust client-controlled claims:** Bind userId/role to a server-managed session (a session store, or a revocable refresh token) instead of relying solely on a long-lived, self-signed JWT, and enforce object-level authorization checks on every endpoint that returns sensitive data.
- **Consider asymmetric signing (RS256/ES256):** This allows the private signing key to remain out of reach of services that only need to verify tokens, reducing the key's exposure surface.
- **Apply standard JWT hardening practices:** short token lifetimes, periodic key rotation, strict issuer/audience validation, and logging/alerting on repeated signature-verification failures (a signal of secret brute-forcing).